Arrays në gjuhën C

Programim në gjuhën C

Alion Emini

kurseinformatike.com

Përmbajtja

1. Hyrje
2. Arrays një-dimensionale
3. Arrays dhe ciklet `for`
4. Inicializimi i array-ve
5. Operatori `sizeof` me arrays
6. Array-t si argumente të funksioneve
7. Ushtrime
8. Përmbledhje

Përmbajtja

1. Hyrje
2. Arrays një-dimensionale
3. Arrays dhe ciklet `for`
4. Inicializimi i array-ve
5. Operatori `sizeof` me arrays
6. Array-t si argumente të funksioneve
7. Ushtrime

Hyrje: nga skalarët te agregatet

Ideja kryesore

Deri tani kemi përdorur variabla **skalare** (mbajnë *një* vlerë). C mbështet edhe variabla **agregate**, që ruajnë *koleksione* vlerash.

Dy agregate në C

- **Arrays**: shumë vlera të *të njëjtit tip*.
- **Struktura** (`struct`): disa vlera me *tipe të ndryshme*.

Fokus i këtij leksioni

Arrays **një-dimensionale** (1D) dhe përdorimi i tyre praktik në programe.

Përmbajtja

1. Hyrje
2. Arrays një-dimensionale
3. Arrays dhe ciklet `for`
4. Inicializimi i array-ve
5. Operatori `sizeof` me arrays
6. Array-t si argumente të funksioneve
7. Ushtrime

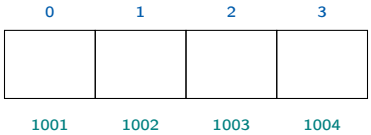
Çfarë është një array?

Përkufizim

Një **array** është një strukturë e të dhënave që përmban një numër elementesh, të gjitha me **të njëjtin tip**. Çdo element identifikohet me **indeks**.

Si e imagjinojmë?

Elementet janë të vendosura njëra pas tjetrës (në memorie, zakonisht në mënyrë të njëpasnjëshme).



Skalar vs. vektorial

Krahasimi

- **Variabla skalare**: mban një vlerë (`char`, `int`, `float`, `double`).
- **Variabla vektoriale**: mban shumë vlera të të njëjtit tip (elemente).

Pse na duhen arrays?

Kur kemi shumë vlera të të njëjtit tip (p.sh. 100 karaktere), është shumë më mirë të përdorim **një array** sesa 100 variabla të veçanta.

Indeksi

Të gjitha elementet ndajnë të njëjtin emër; dallohen vetëm nga **indeksi** (pozicioni).

Deklarimi i një array I

Sintaksa

```
<Tip> <Emer>[<ShprehjeKonstante>];
```

Shembull

```
int a[10];
```

Deklarimi i një array II

Praktikë e mirë: përdorimi i makrove

```c
#define N 10
int a[N];
```

Kujdes

Gjatësia është një **shprehje konstante** (integer) në kohë kompilimi (në këtë nivel).

Indeksimi i array-ve I

Rregulli bazë

Elementet numërohen nga **0**. Nëse gjatësia është **n**, indekset janë **0** deri **n-1**.

Për `int a[10]`

0	1	2	3	4	5	6	7	8	9
a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]	a[8]	a[9]

Indeksimi i array-ve II

a[i] është lvalue

Mund të përdoret si variabël normale:

```c
#include <stdio.h>
#define N 10

int main(void) {
    int a[N] = {0};
    int i = 5;

    a[0] = 1; a[i] = 42;
    ++a[i];

    printf("a[0] = %d\n", a[0]);
    printf("a[5] = %d\n", a[5]);

    return 0;
}
```

Përmbajtja

1. Hyrje
2. Arrays një-dimensionale
3. Arrays dhe ciklet `for`
4. Inicializimi i array-ve
5. Operatori `sizeof` me arrays
6. Array-t si argumente të funksioneve
7. Ushtrime

Arrays dhe `for`: idea bazë

Pse i kombinojmë?

- `for` është ideal kur dimë sa elemente ka array (p.sh. `N`).
- Indeksi `i` ecën nga `0` deri `N-1` dhe akseson `a[i]`.
- Të njëjtin model e ripërdorim për pastrim, lexim, përpunim, kërkim, etj.

Modeli standard

```
for (i = 0; i < 10; i++) { /* punë me a[i] */ }
```

Shembull 1: Pastrimi (inicializimi me 0)

Çfarë bën?

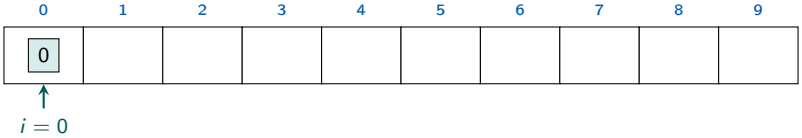
Vendos çdo element të array-t në 0. Kjo është e dobishme kur duam të fillojmë me një gjendje të njohur.

```
for (i = 0; i < N; i++)
    a[i] = 0;
```

Kujdes

Mos përdor `i <= N` sepse `a[N]` është jashtë kufijve.

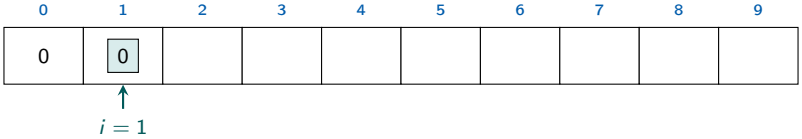
Shembull 1: Hapi 1/10 (i = 0)



Çfarë ndodh në këtë hap?

Ekzekutohet shprehja: `a[i] = 0;` pra `a[0] = 0;`.

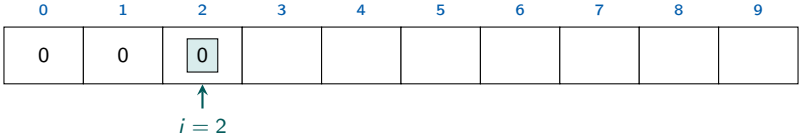
Shembull 1: Hapi 2/10 (i = 1)



Çfarë ndodh në këtë hap?

`a[i] = 0;` pra `a[1] = 0;`.

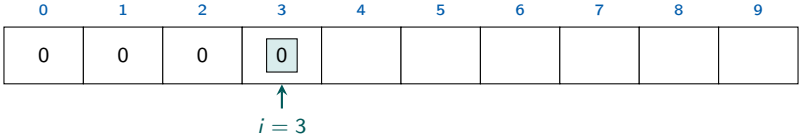
Shembull 1: Hapi 3/10 (i = 2)



Çfarë ndodh në këtë hap?

`a[2] = 0;`.

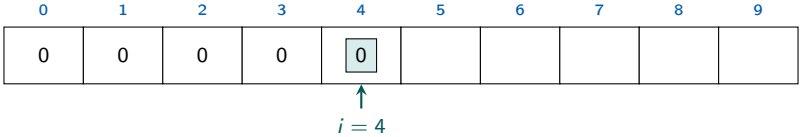
Shembull 1: Hapi 4/10 (i = 3)



Çfarë ndodh në këtë hap?

`a[3] = 0;`.

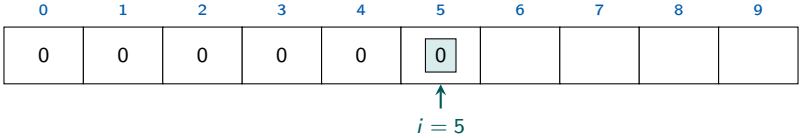
Shembull 1: Hapi 5/10 (i = 4)



Çfarë ndodh në këtë hap?

`a[4] = 0;`.

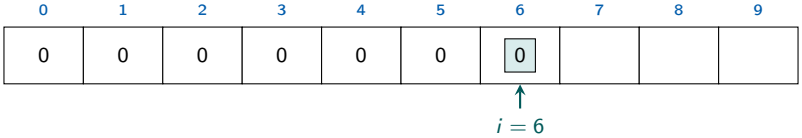
Shembull 1: Hapi 6/10 (i = 5)



Çfarë ndodh në këtë hap?

`a[5] = 0;`.

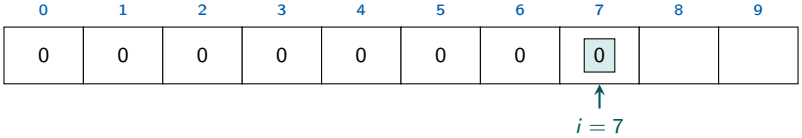
Shembull 1: Hapi 7/10 (i = 6)



Çfarë ndodh në këtë hap?

`a[6] = 0;`.

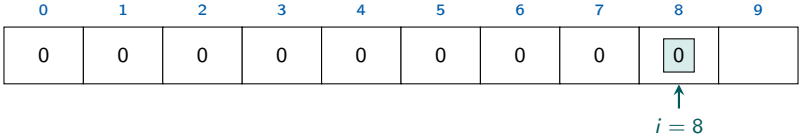
Shembull 1: Hapi 8/10 (i = 7)



Çfarë ndodh në këtë hap?

`a[7] = 0;`.

Shembull 1: Hapi 9/10 (i = 8)



Çfarë ndodh në këtë hap?

`a[8] = 0;`.

Shembull 1: Hapi 10/10 (i = 9)

0	1	2	3	4	5	6	7	8	9
0	0	0	0	0	0	0	0	0	0

$i = 9$

Çfarë ndodh në këtë hap?

`a[9] = 0;` Pastaj bëhet `i++` dhe kontrollohet kushti `i < N`. Kur `i==10`, cikli ndalet.

Shembull 2: Leximi i të dhënave në array

Çfarë bën?

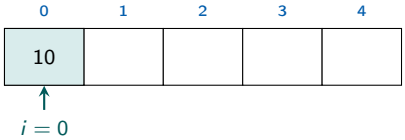
Lexon `N` numra nga input-i dhe i ruan një nga një në `a[i]`.

```
for (i = 0; i < N; i++)
    scanf("%d", &a[i]);
```

Pse `&a[i]`?

`scanf` duhet të dijë **ku** të shkruajë vlerën (adresën në memorie). `&a[i]` jep adresën e elementit.

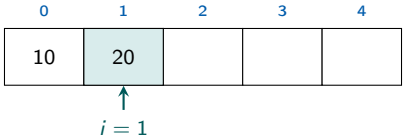
Shembull 2: Hapi 1/5 (lexo a[0])



Çfarë ndodh?

`scanf("%d", &a[i]);` lexon një numër (p.sh. `10`) dhe e ruan në `a[0]`.

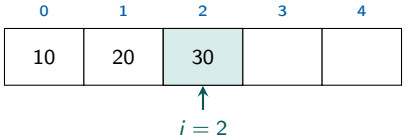
Shembull 2: Hapi 2/5 (lexo a[1])



Çfarë ndodh?

Tani `i=1`, prandaj leximi i radhës ruhet te `a[1]` (p.sh. 20).

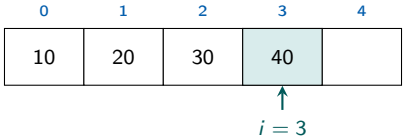
Shembull 2: Hapi 3/5 (lexo a[2])



Çfarë ndodh?

Pas dy leximeve, tani plotësohet edhe a[2] (p.sh. 30).

Shembull 2: Hapi 4/5 (lexo a[3])



Çfarë ndodh?

Cikli vazhdon: vlera e radhës ruhet te a[3] (p.sh. 40).

Shembull 2: Hapi 5/5 (lexo a[4])

0	1	2	3	4
10	20	30	40	50

$i = 4$

Çfarë ndodh?

Pas 5 leximeve, kemi plotësuar `a[0]`..`a[4]`. Cikli vazhdon derisa `i == N`.

Shembull 3: Mbledhja e elementeve (sum)

Çfarë bën?

Llogarit shumën e të gjithë elementeve. Ky është shembull i **akumulimit** (aggregation).

```
sum = 0;
for (i = 0; i < N; i++)
    sum += a[i];
```

Teori: akumulatori

`sum` duhet inicializuar para ciklit; brenda ciklit i shtojmë kontributin e çdo `a[i]`.

Shembull 3: Hapi 1/5 (i = 0)

0	1	2	3	4
10	20	30	40	50

$i = 0$

```
sum = 0 + 10 = 10
```

Shembull 3: Hapi 2/5 (i = 1)

0	1	2	3	4
10	20	30	40	50

$i = 1$

```
sum = 10 + 20 = 30
```

Shembull 3: Hapi 3/5 (i = 2)

0	1	2	3	4
10	20	30	40	50

$i = 2$

```
sum = 30 + 30 = 60
```

Shembull 3: Hapi 4/5 (i = 3)

0	1	2	3	4
10	20	30	40	50

$i = 3$

`sum = 60 + 40 = 100`

Shembull 3: Hapi 5/5 (i = 4)

0	1	2	3	4
10	20	30	40	50

$i = 4$

`sum = 100 + 50 = 150`

Përfundim

Pas përpunimit të të gjithë elementeve, `sum` përmban shumën totale.

Kufijtë: gabimi klasik "një njësi jashtë" I

Në C nuk ka kontroll automatik kufijsh

Indeksi jashtë kufijve ⇒ **sjellje e papërcaktuar** (*undefined behavior*).

```
int a[10], i;
for (i = 1; i <= 10; i++)
    a[i] = 0;
```

Çfarë ndodh?

Kur `i==10`, programi shkruan në `a[10]` (që nuk ekziston). Mund të krijojë efekte të çuditshme, crash, ose rezultate të gabuara.

Kufijtë: gabimi klasik "një njësi jashtë" II

Saktë

```c
for (i = 0; i < 10; i++)
    a[i] = 0;
```

Kujdes me efektet anësore në indeks

Rrezik: lexon dhe ndryshon indeksin në të njëjtën shprehje

```
i = 0;
while (i < N)
    a[i] = b[i++];
```

Zgjidhje

Hiq efektin anësor nga indeksi; përdor një `for` të pastër:

```
for (i = 0; i < N; i++)
    a[i] = b[i];
```

Programi 1: Radhitja në rend të kundërt (koncept)

Qëllimi

Lexo `N` numra, ruaji në array, pastaj shfaqi në rend të kundërt (pa i ndryshuar fizikisht).

Shembull

```
Futni 10 numra:  34 82 49 102 7 94 23 11 50 31
Në rend të kundërt:  31 50 11 23 94 7 102 49 82 34
```

Përfitimi i makros `N`

Ndryshon vetëm `#define N ...` dhe programi përshtatet automatikisht.

Programi 1: Paraqitja grafike e array-t përpara printimit mbrapsht

Array a pasi lexohen `N=10` numra

0	1	2	3	4	5	6	7	8	9
34	82	49	102	7	94	23	11	50	31

Programi 1: Kodi

```c
#include <stdio.h>
#define N 10

int main(void) {
    int a[N], i;

    printf("Futni %d numra: ", N);
    for (i = 0; i < N; i++)
        scanf("%d", &a[i]);

    printf("Në rend të kundërt:");
    for (i = N - 1; i >= 0; i--)
        printf(" %d", a[i]);
    printf("\n");

    return 0;
}
```

Programi 1: Paraqitja grafike e array-t së printuar mbrapsht

Array a që do të printohet me ciklin for

0	1	2	3	4	5	6	7	8	9
31	50	11	23	94	7	102	49	82	34

Përmbajtja

1. Hyrje
2. Arrays një-dimensionale
3. Arrays dhe ciklet `for`
4. Inicializimi i array-ve
5. Operatori `sizeof` me arrays
6. Array-t si argumente të funksioneve
7. Ushtrime

Inicializimi: lista me vlera

Inicializuesi tipik

```
1 int a[10] = {1,3,5,2,4,7,6,9,8,0};
```

0	1	2	3	4	5	6	7	8	9
1	3	5	2	4	7	6	9	8	0

Rregull

Nëse inicializuesi është më i shkurtër, pjesa e mbetur inicializohet me **0**.

Inicializimi i pjesshëm dhe me zero

Inicializim i pjesshëm

```
int a[10] = {1,2,3,4,5,6};
```

0	1	2	3	4	5	6	7	8	9
1	2	3	4	5	0	0	0	0	0

Të gjitha zero

```
int a[10] = {0};
```

0	1	2	3	4	5	6	7	8	9
0	0	0	0	0	0	0	0	0	0

Gjatesia mund të lihet bosh

Nëse inicializuesi ekziston, gjatësia mund të mos shkruhet

```
int a[] = {1,2,3,4,5,6,7,8,9,10};
```

0	1	2	3	4	5	6	7	8	9
1	2	3	4	5	6	7	8	9	10

Çfarë bën kompilatori?

Përcakton gjatësinë nga numri i elementeve në inicializues.

Programi 2: Ideja

Qëllimi

Kontrollo nëse një numër ka ndonjë shifër të përsëritur.

Shembull

```
Futni një numër:  28212
Shifër e përsëritur
```

Strategjia

Përdorim një array `digit_seen[10]` (indeks 0..9) për të shënuar shifrat e hasura.

Programi 2: Kodi I

```c
#include <stdbool.h>
#include <stdio.h>

int main(void) {
    bool digit_seen[10] = {false};
    int digit;
    long n;

    printf("Futni një numër: ");
    scanf("%ld", &n);
```

Programi 2: Kodi II

```
    while (n > 0) {
        digit = (int)(n % 10);
        if (digit_seen[digit])
            break;
        digit_seen[digit] = true;
        n /= 10;
    }

    if (n > 0)
        printf("Shifër e përsëritur\n");
    else
        printf("Asnjë shifër e përsëritur\n");

    return 0;
}
```

Programi 2: Kodi III

Shënim (nëse s'ke <stdbool.h>)

```
/* Alternativë minimale */
#define true 1
#define false 0
typedef int bool;
```

Programi 2: Idea vizuale (`digit_seen[10]`)

Çfarë ruan `digit_seen`?

`digit_seen[d] = true` do të thotë: shifra `d` është parë të paktën një herë.

0	1	2	3	4	5	6	7	8	9
false	false	false	false	false	false	false	false	false	false

Fillimi

Në start, të gjitha janë `false` (asnjë shifër s'është parë ende).

Programi 2: Hapi 1 (lexo shifrën 2)

0	1	2	3	4	5	6	7	8	9
false	false	true	false	false	false	false	false	false	false

Çfarë ndodh?

`digit = 2`. Meqenëse `digit_seen[2]` është `false`, e vendosim `true`.

Programi 2: Hapi 2 (lexo shifrën 1)

0	1	2	3	4	5	6	7	8	9
false	true	true	false	false	false	false	false	false	false

Çfarë ndodh?

`digit = 1`. Siqë `digit_seen[1]` është `false`, e vendosim `true`.

Programi 2: Hapi 3 (lexo shifrën 2 përsëri)

0	1	2	3	4	5	6	7	8	9
false	true	true	false	false	false	false	false	false	false

Përfundim

Tani `digit_seen[2]` është tashmë `true` ⇒ shifra 2 është e përsëritur, prandaj cikli ndalet (`break`).

Shembulli 4: Lexo karaktere dhe printo në rend të kundërt

```c
#include <stdio.h>
#define N 100

int main(void) {
    char c, buf[N];
    int i;

    for (i = 0; i < N && (c = (char)getchar()) != '\n'; i++)
        buf[i] = c;

    for (i = i - 1; i >= 0; i--)
        printf("%c", buf[i]);
    printf("\n");

    return 0;
}
```

Kujdes

Indeksi në printim fillon me `i-1`, sepse pas leximit `i` tregon **numrin e karaktereve** të lexuar.

Shembulli 4: Hapi 1/3 (mbushja e `buf`)

Ideja

Karakteret ruhen radhazi në `buf[0]`, `buf[1]`, ... derisa të lexohet `'\n'`.

0	1	2	3	4
H	E	L	L	O

$i = 5$

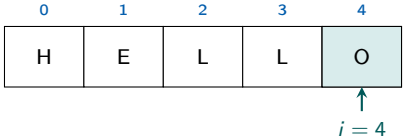
Rezultat

Pas leximit të `HELLO`, `i` tregon numrin e karaktereve të ruajtura (këtu 5).

Shembulli 4: Hapi 2/3 (fillimi i printimit mbrapsht)

Pse `i-1`?

Kur mbaron leximi, `i` është numërimi i karaktereve, prandaj karakteri i fundit është te `buf[i-1]`.



Output (deri tani)

```
0
```

Shembulli 4: Hapi 3/3 (output final)

0	1	2	3	4
H	E	L	L	O

Përfundim

Cikli i dytë printon: `buf[4]`, `buf[3]`, `buf[2]`, `buf[1]`, `buf[0]`.

Output

```
OLLEH
```

Shembulli 5: Numërimi i karaktereve ASCII I

Ideja

code[c] mban sa herë është lexuar karakteri me kodin ASCII c.

```c
#include <stdio.h>
#define N 128
#define S 32

int main(void) {
    int c, code[N] = {0};

    while ((c = getchar()) != EOF)
        if (c >= 0 && c < N) code[c]++;

    for (c = S; c < N; c++)
        if (code[c] > 0) printf("Kodi: %3d Simboli: %c Herë: %d\n", c, c, code[c]);

    return 0;
}
```

Shembulli 5: Numërimi i karaktereve ASCII II

Shënim

`EOF` është makro standarde (zakonisht `-1`). Mos e rishkruaj me `#define`.

Përmbajtja

1. Hyrje
2. Arrays një-dimensionale
3. Arrays dhe ciklet `for`
4. Inicializimi i array-ve
5. Operatori `sizeof` me arrays
6. Array-t si argumente të funksioneve
7. Ushtrime

sizeof për gjatësinë e array-t

Formula tipike

```
sizeof(a) / sizeof(a[0])
```

Përdorim praktik

```
for (i = 0; i < (int)(sizeof(a) / sizeof(a[0])); i++)
    a[i] = 0;
```

Pse kasti në int?

sizeof kthen size_t (tip i panënshkruar). Kasti shmang paralajmërime në disa kompilatorë.

Makro për "SIZE" (shembull)

```c
#define SIZE(a) ((int)(sizeof(a) / sizeof((a)[0])))

for (i = 0; i < SIZE(a); i++)
    a[i] = 0;
```

Avantazh

Makroja merr array-n si parametër dhe punon për çdo array statik (jo pointer).

Programi 3: Qëllimi

Qëllimi

Printon një tabelë të interesit të përbërë për një investim (p.sh. 100), për disa norma interesi dhe disa vite.

Input

Norma fillestare (p.sh. 6) dhe numri i viteve (p.sh. 5).

Ide

`value[i]` ruan vlerën aktuale për normën `low_rate + i`.

Programi 3: Kodi I

```c
/* Printon një tabelë të interesit të përbërë */
#include <stdio.h>

#define INITIAL_BALANCE 100.00

int main(void) {
    int i, low_rate, num_years, year;
    double value[5];
    int num_rates = (int)(sizeof(value) / sizeof(value[0]));

    printf("Futni normën e interesit: ");
    scanf("%d", &low_rate);
    printf("Futni numrin e viteve: ");
    scanf("%d", &num_years);
```

Programi 3: Kodi II

```
    printf("\nVitet");
    for (i = 0; i < num_rates; i++) {
        printf("%8d%%", low_rate + i);
        value[i] = INITIAL_BALANCE;
    }
    printf("\n");
```

Programi 3: Kodi III

```c
    for (year = 1; year <= num_years; year++) {
        printf("%4d", year);
        for (i = 0; i < num_rates; i++) {
            value[i] += (low_rate + i) / 100.0 * value[i];
            printf("%9.2f", value[i]);
        }
        printf("\n");
    }

    return 0;
}
```

Programi 3: Kodi IV

Çfarë po ndodh?

- **Array** `value`: vlera e investimit për çdo normë.
- **Cikli i jashtëm**: vitet.
- **Cikli i brendshëm**: përditëson dhe printon `value[i]`.

Përmbajtja

1. Hyrje
2. Arrays një-dimensionale
3. Arrays dhe ciklet `for`
4. Inicializimi i array-ve
5. Operatori `sizeof` me arrays
6. Array-t si argumente të funksioneve
7. Ushtrime

Ideja kryesore

Çfarë ndodh kur e kalojmë një array në funksion?

Në C, kur një array kalohet si parametër, ai **"zbërthehet" (decays)** në **pointer** te elementi i parë.

Pasojë

Brenda funksionit `sizeof(a)` **nuk** jep madhësinë e plotë të array-t (sepse a është pointer).

Sintaksa e parametrave

Tre forma ekuivalente

```
1 void f(int a[]);
2 void f(int a[10]);
3 void f(int *a);
```

E rëndësishme

Të treja janë ekuivalente për parametrat: kompilatori i trajton si `int *a`.

Gjithmonë kalo edhe gjatësinë

Modeli standard

```c
int sum(const int a[], int n) {
    int s = 0;
    for (int i = 0; i < n; i++)
        s += a[i];
    return s;
}
```

Pse `const`?

Nëse funksioni **nuk** duhet ta ndryshojë array-n, përdor `const` për siguri.

Shembull: funksion që e ndryshon array-n

Pastrimi me 0

```c
void clear(int a[], int n) {
    for (int i = 0; i < n; i++)
        a[i] = 0;
}
```

Shënim

Ndryshimet brenda funksionit reflektohen jashtë (sepse punojmë me të njëjtën memorie).

Përmbajtja

1. Hyrje
2. Arrays një-dimensionale
3. Arrays dhe ciklet `for`
4. Inicializimi i array-ve
5. Operatori `sizeof` me arrays
6. Array-t si argumente të funksioneve
7. Ushtrime

Ushtrimi 1: Array i përmbysur

Detyra

Shkruani një program që përmbys renditjen e elementeve të një array të numrave të plotë. Përdorni një variabël `temp`.

Ushtrimi 1: Zgjidhja

```c
#include <stdio.h>
#define N 5

int main(void) {
    int array[N] = {1, 2, 3, 4, 5};
    int temp, i;

    for (i = 0; i < N / 2; i++) {
        temp = array[i];
        array[i] = array[N - i - 1];
        array[N - i - 1] = temp;
    }

    printf("Array i përmbysur: ");
    for (i = 0; i < N; i++)
        printf("%d ", array[i]);
    printf("\n");

    return 0;
}
```

Ushtrimi 2: Ndërrimi i dy elementeve

Detyra

Ndërroni vendin e dy elementeve të një array. Programi:

- printon array-n fillestar,
- lexon vlerën 1 dhe gjen pozicionin,
- lexon vlerën 2 dhe gjen pozicionin,
- bën ndërrimin dhe printon rezultatin.

Ushtrimi 2: Zgjidhja I

```c
#include <stdio.h>
#define N 5

int main(void) {
    int array[N] = {10, 20, 30, 40, 50};
    int vlera1, vlera2, poz1 = -1, poz2 = -1, temp, i;

    printf("Array fillestar: ");
    for (i = 0; i < N; i++) printf("%d ", array[i]);
    printf("\n");

    printf("Futni vlerën e parë për të ndërruar: ");
    scanf("%d", &vlera1);
    for (i = 0; i < N; i++) {
        if (array[i] == vlera1) { poz1 = i; break; }
    }

    printf("Futni vlerën e dytë për të ndërruar: ");
    scanf("%d", &vlera2);
```

Ushtrimi 2: Zgjidhja II

```c
    for (i = 0; i < N; i++) if (array[i] == vlera2) { poz2 = i; break; }

    if (poz1 == -1 || poz2 == -1 || vlera1 == vlera2) {
        printf("Gabim: Vlerat nuk janë të vlefshme ose janë të njëjta.\n");
        return 1;
    }

    temp = array[poz1];
    array[poz1] = array[poz2];
    array[poz2] = temp;

    printf("Array pas ndërrimit: ");
    for (i = 0; i < N; i++) printf("%d ", array[i]);
    printf("\n");

    return 0;
}
```

Ushtrimi 3: Krahasimi i dy array-ve

Detyra

Shkruani një program që krahason dy array të numrave të plotë dhe përcakton nëse janë identike (të njëjtat vlera në të njëjtat pozicione).

Ushtrimi 3: Zgjidhja I

```c
#include <stdio.h>
#include <stdbool.h>

#define N 5

bool janeTeBarabarte(int a[], int b[], int n) {
    for (int i = 0; i < n; i++) {
        if (a[i] != b[i]) return false;
    }
    return true;
}
```

Ushtrimi 3: Zgjidhja II

```c
int main(void) {
    int array1[N] = {1, 2, 3, 4, 5};
    int array2[N] = {1, 2, 3, 4, 5};

    if (janeTeBarabarte(array1, array2, N))
        printf("Array-t janë të njëjta.\n");
    else
        printf("Array-t nuk janë të njëjta.\n");

    return 0;
}
```

Përmbajtja

1. Hyrje
2. Arrays një-dimensionale
3. Arrays dhe ciklet `for`
4. Inicializimi i array-ve
5. Operatori `sizeof` me arrays
6. Array-t si argumente të funksioneve
7. Ushtrime

Përmbledhje

Çfarë mësuam?

- Çfarë është një array 1D dhe pse përdoret.
- Deklarimi, indeksimi (0..n-1) dhe përdorimi me `for`.
- Inicializimi me lista dhe rregullat e zero-inicializimit.
- Shembuj praktikë: rend i kundërt, shifra të përsëritura, përpunim karakteresh, `sizeof`.

Këshillë

Shmang indeksimin jashtë kufijve dhe mbaj ciklet të pastra (pa efekte anësore në indekse).

Faleminderit!

Pyetje?